

[CS230] Structuring Machine Learning Projects: Transfer Learning & Multi-task Learning

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-6. ML Strategy Basics - *Completed*

Chapter 7. Advanced Learning Strategies (Current Unit)

- 7.1 Error Analysis - *Completed*
- 7.2 Data Mismatch - *Completed*
- **7.3 Transfer Learning & Multi-task Learning**
 - * Concept: Standing on the Shoulders of Giants
 - * Fine-tuning Strategies (Freeze vs Unfreeze)
 - * Multi-task Learning (Shared Representation)
 - * Implementation: Keras Code

Chapter 8. End-to-End Deep Learning - *Upcoming*

지난 시간 복습 및 연결

지금까지 우리는 모델을 밑바닥부터(Scratch) 학습시키는 법을 배웠습니다. 하지만 현실 세계의 엔지니어는 "아무것도 없는 상태에서 시작하지 않습니다." 책을 읽을 때마다 '가나다라'부터 다시 배우지 않듯, AI 모델도 남이 이미 학습해둔 지식(Knowledge)을 빌려와서 자신의 문제를 해결할 수 있습니다. 이것이 **전이 학습(Transfer Learning)**이며, 현대 딥러닝 성공의 90%는 여기에 기인합니다.

1 Unit Overview

□ 핵심 목표

이 단원은 '데이터 부족'을 해결하고 학습 효율을 극대화하는 두 가지 패러다임을 다룹니다.

- **Transfer Learning:** 대규모 데이터(ImageNet)로 학습된 모델을 가져와 내 문제(X-ray)에 적용하는 법을 배웁니다.
- **Fine-tuning:** 가중치를 고정(Freeze)하거나 미세 조정(Unfreeze)하는 단계별 전략을 익힙니다.
- **Multi-task Learning:** 하나의 모델이 여러 작업을 동시에 수행하며 지능을 높이는 원리를 이해합니다.
- **구현:** Keras를 활용해 Pre-trained Model을 로드하고 커스터마이징하는 코드를 작성합니다.

2 Essential Terminology

용어	의미	비유
Transfer Learning	지식 전이 ($A \rightarrow B$)	영어 잘하는 사람이 불어도 빨리 배움.
Pre-trained Model	사전 학습된 모델 (Source)	이미 박사 학위를 받은 전문가.
Fine-tuning	미세 조정	전문가에게 우리 회사의 업무 매뉴얼만 가르침.
Multi-task Learning	동시 학습 ($A \& B$)	수학과 물리를 동시에 배우면 시너지가 남.

3 Core Concepts: 지식의 재활용

3.1 1. Transfer Learning (전이 학습)

데이터가 풍부한 Task A(Source)에서 배운 지식을 데이터가 적은 Task B(Target)로 옮깁니다.

- **앞단 (Early Layers):** 엣지, 곡선, 질감 등 보편적인 특징을 배웁니다. (재활용 가능)
- **뒷단 (Later Layers):** 구체적인 사물(고양이, 자동차)을 배웁니다. (새로 학습 필요)

3.2 2. Multi-task Learning (다중 작업 학습)

하나의 신경망이 여러 작업(Task A, B, C)을 동시에 수행합니다.

- **Shared Layers:** 모든 작업에 공통적으로 필요한 저수준 특징(Low-level features)을 공유합니다.
- **효과:** 서로 다른 작업들이 일종의 노이즈(Regularization) 역할을 하여 과대적합을 막고 일반화 성능을 높입니다.
- **예시:** 자율주행 (표지판 인식 + 신호등 인식 + 보행자 감지).

4 Deep Dive: Fine-tuning Strategies

”데이터가 얼마나 있느냐”에 따라 전략이 달라집니다.

□ 데이터 규모별 전략 (전략 가이드)

1. Small Data (데이터 매우 적음):

- 전략: Backbone 전체 고정 (**Freeze**).
- 행동: 마지막 분류기(Head)만 떼어내고 새로 학습시킵니다.

2. Medium Data (적당함):

- 전략: 앞단 일부 고정, 뒷단 일부 해제 (**Fine-tuning**).
- 행동: 상위 층(Later Layers)의 가중치를 미세하게 업데이트합니다.

3. Big Data (데이터 많음):

- 전략: 전체 재학습 (**Retrain All**).
- 행동: 사전 학습된 가중치를 초기값(Initialization)으로만 쓰고 전체를 다 학습합니다.

5 Implementation: Transfer Learning with Keras

MobileNetV2를 가져와서 커스텀 분류기를 만드는 코드입니다.

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, models
3
4 def build_transfer_model(input_shape, num_classes):
5     # 1. Load Pre-trained Model (MobileNetV2)
6     # include_top=False: 개1000 클래스 분류기(Head)는 버림
7     # weights='imagenet': 으로ImageNet 학습된 가중치 사용
8     base_model = tf.keras.applications.MobileNetV2(
9         input_shape=input_shape,
10        include_top=False,
11        weights='imagenet'
12    )
13
14    # 2. Freeze the Base Model 핵심(!)
15    # 역전파 시 이 모델의 가중치는 변하지 않도록 잠금
16    base_model.trainable = False
17
18    # 3. Add Custom Head
19    model = models.Sequential([
20        base_model,
21        layers.GlobalAveragePooling2D(), # 특징맵을 벡터로 변환
22        layers.Dropout(0.2),             # 과대적합 방지
23        layers.Dense(num_classes, activation='softmax') # 내 문제에 맞는 출력층
24    ])
25
26    return model
27
28 # --- 실행 및 Fine-tuning ---
29 if __name__ == "__main__":
30     model = build_transfer_model((160, 160, 3), 10)
31
32     # 단계1: 만Head 학습 (는Base 고정)
```

```

33 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['acc'
34 ])
35 # model.fit(...)
36 # 단계2: Fine-tuning (의Base 일부를 풀)
37 base_model = model.layers[0]
38 base_model.trainable = True
39
40 # 앞쪽 개100 층은 계속 고정, 나머지 뒤쪽만 학습
41 fine_tune_at = 100
42 for layer in base_model.layers[:fine_tune_at]:
43     layer.trainable = False
44
45 # 중요: 미세 조정 시에는 학습률을 아주 낮게 (1/10 ~ 1/100) 잡아야 함
46 model.compile(optimizer=tf.keras.optimizers.RMSprop(lr=1e-5),
47               loss='categorical_crossentropy', metrics=['acc'])
48 # model.fit(...)

```

Listing 1: Transfer Learning Pipeline

6 FAQ & Pitfalls

처음부터 Fine-tuning을 하면 안 되나요? (오해 방지 가이드)

절대 안 됩니다. 새로 붙인 Head(분류기)는 랜덤 초기화 상태라 엉뚱한 오차(Gradient)를 뿜어냅니다. Base Model을 고정하지 않으면, 이 큰 오차 때문에 잘 학습되어 있던 Base Model의 가중치가 다 망가져버립니다 (**Catastrophic Forgetting**). 반드시 **Head**를 먼저 학습시켜 안정화한 뒤, Base Model을 풀어야 합니다.

Q. 입력 이미지 크기가 달라도 되나요?

A. CNN의 특성상 가능은 하지만, 성능을 위해 사전 학습 모델이 사용했던 크기(예: 224x224)로 리사이징(Resize)해서 넣는 것을 권장합니다.

다음 단계 (Next Step)

이것으로 '머신러닝 프로젝트 구조화 전략' 파트를 마칩니다. 여러분은 이제 데이터를 다루는 전략부터, 남의 지식을 빌려오는 전략까지 모두 갖춘 전략가가 되었습니다.

다음 챕터부터는 딥러닝을 더욱 깊이 있게 만드는 **[End-to-End Deep Learning]**의 개념과, 이것이 전통적인 파이프라인 방식과 어떻게 다른지 비교 분석하며 시작해 보겠습니다. 수고하셨습니다.

□ 단원 요약 (Cheat Sheet)

1. **Transfer Learning:** 빅데이터 모델(Source)을 소량 데이터 문제(Target)에 재활용한다.
2. **Freeze:** 초기 학습 시에는 Backbone을 고정하고 Head만 학습한다.
3. **Fine-tune:** 데이터가 충분하면 Backbone의 일부를 풀어 미세 조정한다. (Low LR 필수)
4. **Multi-task:** 여러 작업을 동시에 배우면 일반화 성능이 좋아진다.